

OSSP-WEEK4-SESSION4

Task1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Source code:

```
GNU nano 7.2 task1.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t child[3];
    int status;

    for (int i = 0; i < 3; i++)
    {
        child[i] = fork();

        if (child[i] < 0)
        {
            perror("fork failed");
            exit(1);
        }

        if (child[i] == 0)
        {
            printf("Child %d: PID = %d, PPID = %d\n",
                i + 1, getpid(), getppid());

            sleep(i + 1);
            sleep(1 + i);

            printf("Child %d completed.\n", i + 1);
            exit(10 + i);
        }
    }

    printf("\nParent: Waiting for any child using wait()...\n");

    pid_t pid = wait(&status);

    if (pid > 0 && WIFEXITED(status))
    {
        printf("wait(): Child with PID %d terminated with status %d\n",
            pid, WEXITSTATUS(status));
    }

    printf("\nParent: Waiting for specific children using waitpid()...\n");

    for (int i = 0; i < 3; i++)
    {
        pid = waitpid(child[i], &status, 0);

        if (pid > 0 && WIFEXITED(status))
        {
            printf("waitpid(): Child with PID %d terminated with status %d\n",
                pid, WEXITSTATUS(status));
        }
    }

    printf("\nParent process completed.\n");

    return 0;
}
```

Output:

```
PS C:\Users\SAIRAM0806> wsl

sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ cd /mnt/c/Users/SAIRAM0806/task1
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806/task1$ nano task1.c
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806/task1$ nano task1.c
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806/task1$ ./task1
Child 1: PID = 1566, PPID = 1565
Child 2: PID = 1567, PPID = 1565

Parent: Waiting for any child using wait()...
Child 3: PID = 1568, PPID = 1565
Child 1 completed.
wait(): Child with PID 1566 terminated with status 10

Parent: Waiting for specific children using waitpid()...
Child 2 completed.
waitpid(): Child with PID 1567 terminated with status 11
Child 3 completed.
waitpid(): Child with PID 1568 terminated with status 12

Parent process completed.
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806/task1$ |
```

Observation:

I learned how a parent process can create multiple child processes using `fork()` and synchronize their completion using `wait()` and `waitpid()`. I understood that `wait()` waits for any child process, while `waitpid()` can wait for a specific child process. I also learned how to check the child's termination status using `WIFEXITED()` and `WEXITSTATUS()`.

Task2:

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

Source code:

```
GNU nano 7.2 zombie.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child PID: %d\n", getpid());
        printf("Child exiting...\n");
        exit(0);
    }

    printf("Parent is sleeping without calling wait().\n");
    printf("Child becomes a zombie after termination.\n");

    sleep(20);

    printf("Parent exiting.\n");

    return 0;
}
```

Output:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\SAIRAM0806> wsl
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ nano zombie.c
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ gcc zombie.c -o zombie
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ ./zombie
Parent PID: 1757
Parent is sleeping without calling wait().
Child becomes a zombie after termination.
Child PID: 1758
Child exiting...
Parent exiting.
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ |
```

```
PS C:\Users\SAIRAM0806> wsl
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ ps -ef | grep defunct
sairam0+  1825    1824  0 08:57 pts/4    00:00:00 [practical_week4] <defunct>
sairam0+  1845    1831  0 08:57 pts/0    00:00:00 grep --color=auto defunct
sairam0806@LAPTOP-M6NQUIAP:/mnt/c/Users/SAIRAM0806$ |
```

Observation:

The child process terminates before the parent process and remains in the process table as a zombie because the parent has not called `wait()` or `waitpid()`. The zombie process is identified by the status in the process table.